

AMCAS - Assignment 1

Name: Nikhil Venkat Atkuru Roll No: 2024102039

Part A

Task 1

The simulated bitline differential $\Delta V(BL, BLB)$ at $t = 2$ ns is **729.9 mV**, which is **10.58 $\times$ larger** than the 69 mV reference value from Lecture 1.

Task 2

During a read access, the access transistor connected to node q ('0') gets charged by the precharged bitline and is shown like the bump in the plot and discharges again down due to the pull down transistor.

Configuration	Access Width ($W_{MA1,MA2}$)	ΔV ($t = 2$ ns)	Peak $v(q)$ (q_{max})
Original Baseline	0.16 μm	729.9 mV	205.5 mV
Widened Access	0.24 μm	852.3 mV	271.2 mV

Table 1: Access transistor sizing impact on discharge and read disturbance.

Widening access transistors speeds up discharge ($\Delta V = 852.3$ mV) but degrades the cell ratio ($\beta = W_{\text{driver}}/W_{\text{access}}$) from 1.25 to 0.833, elevating peak $v(q)$ to 271.2 mV and severely compromising read stability.

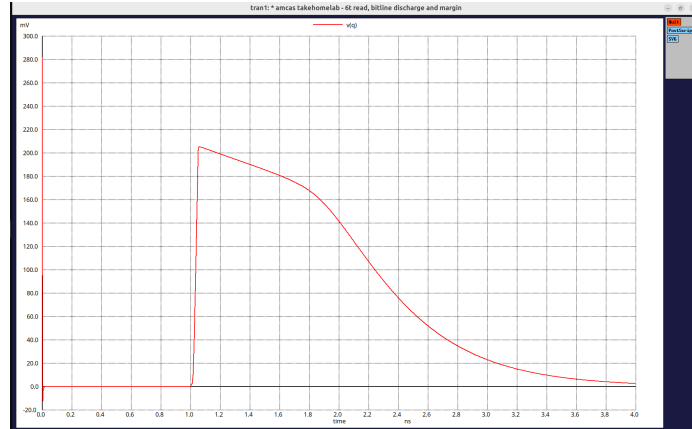


Figure 1: Transient waveform of internal node voltage $v(q)$.

Task 3

V_{DD} was swept from 1.1 V to 0.6 V in 50 mV steps.

V_{DD} (V)	ΔV (mV)	V_{DD} (V)	ΔV (mV)	V_{DD} (V)	ΔV (mV)
1.10	729.9	0.90	497.4	0.70	257.1
1.05	673.6	0.85	437.2	0.65	199.1
1.00	616.0	0.80	376.8	0.60	143.9
0.95	557.1	0.75	316.6	—	—

Table 2: Bitline differential ΔV at $t = 2$ ns vs. V_{DD} .

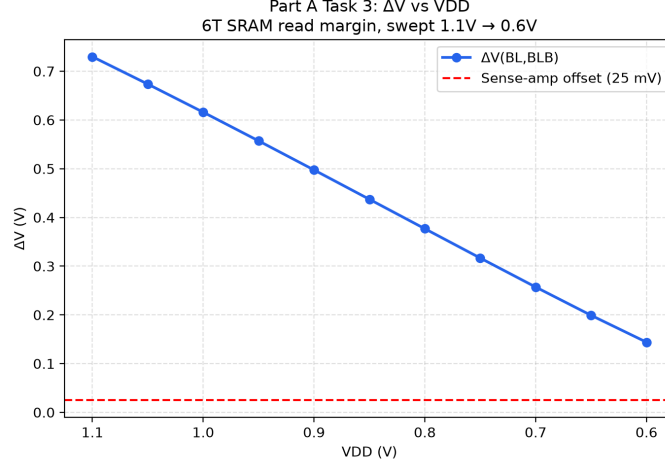


Figure 2: ΔV vs. V_{DD} relative to 25 mV sense-amp offset.

At $V_{DD} = 0.60$ V, $\Delta V = 143.9$ mV remains well above the 25 mV threshold. Extrapolating the ~ 1.17 mV/mV slope, failure occurs at $V_{DD} \approx 0.495$ V.

Task 4

Metric	27°C (Task 1)	85°C (Task 4)	Change
$\Delta V(BL, BLB)$ at $t = 2$ ns	729.9 mV	582.1 mV	-20.25%
$q_{\max}$ (Peak $v(q)$)	205.5 mV	220.8 mV	+7.45%

Table 3: Thermal impact (27°C vs. 85°C).

At 85°C, carrier mobility degradation reduces I_{read} , dropping ΔV by 20.25%. Because sense-amp offset drifts by only a few millivolts, discharge margin degradation is the primary driver of high-temperature failure.

Part B - CACTI: Size the 2 MB SRAM L2

Task 1

Metric	Baseline Value
Access Time	2.902 ns
Dynamic Read Energy	0.7929 nJ (792.89 pJ)
Standby Leakage Power (per bank)	562.63 mW
Total Cache Area	11.47 mm ² (2.213 mm $\times$ 5.184 mm)

Table 4: Baseline 2 MB SRAM L2 cache metrics (45 nm ITRS-HP, ED^2).

Task 2

Capacity was swept from 256 kB to 16 MB (4 banks, 64 B blocks).

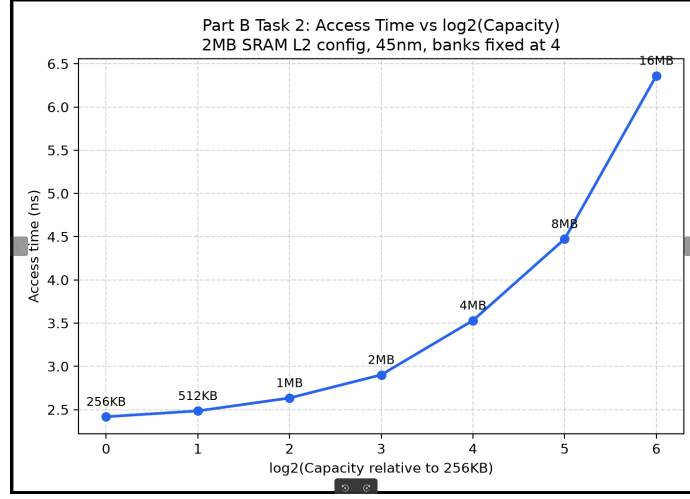


Figure 3: Access time vs. $\log_2(\text{Capacity})$ for 45 nm SRAM (4 UCA banks).

- **256 kB to 2 MB:** Delay scales linearly (≈ 0.16 ns per doubling), matching Amrutur & Horowitz’s FO4 delay model.
- **2 MB to 16 MB:** Delay scales super-linearly (1.88 ns jump from 8 MB to 16 MB). Fixed bank count forces larger sub-arrays, causing global H-tree RC delays to dominate.

Task 3

Metric / Parameter	Pure Delay	Pure Area	ED^2P Balanced
Objective Weight	100:0:0:0	0:0:0:100	0:0:0:100:0
Data Triplet ($N_{dwl}, N_{dbl}, N_{spd}$)	4, 2, 1	2, 2, 1	4, 2, 1
Access Time	2.892 ns	4.231 ns	2.902 ns
Dynamic Read Energy	0.898 nJ	0.790 nJ	0.793 nJ
Leakage Power (per bank)	560.77 mW	548.28 mW	562.63 mW
Total Area	10.21 mm ²	8.63 mm ²	10.66 mm ²
Data Area Efficiency	56.83%	67.69%	54.33%

Table 5: Array configurations across optimization targets.

Trade-offs:

- **Pure Area:** $N_{dwl} = 2, N_{dbl} = 2$. Fewer sub-arrays boost area efficiency (67.69%), but longer wires degrade access time (4.231 ns).
- **Pure Delay:** $N_{dwl} = 4, N_{dbl} = 2$. Partitioning cuts RC delay (2.892 ns) at the cost of higher energy (0.898 nJ).
- **ED^2P :** Retains fast array layout ($N_{dwl} = 4, N_{dbl} = 2$) but scales down sense-amp drivers, achieving near-optimal delay (2.902 ns) with 11.7% less dynamic energy (0.793 nJ).

Task 4

Hand Calculation: For a 256-cell bitline ($L \approx 120 \mu\text{m}$, $R_L \approx 42 \Omega$, $C_L \approx 180 \text{ fF}$):

$$t_{\text{hand}} = 0.38 \cdot R_L \cdot C_L = 0.38 \cdot (42 \Omega) \cdot (180 \text{ fF}) \approx 2.87 \text{ ps} \quad (1)$$

Adding access transistor resistance ($R_{\text{ON}} \approx 8 \text{ k}\Omega$): $t_{\text{disch}} \approx 0.69 \cdot R_{\text{ON}} \cdot C_L \approx 993.6 \text{ ps}$.

Discrepancy vs. CACTI (406.9 ps): CACTI models differential sense-amp triggering threshold ($\Delta V_{\text{sa}} \approx 50\text{--}100 \text{ mV}$) rather than full/half swing, precharge switch resistances, and non-linear BSIM4 transistor overlap parasitics (C_{gd}).

Part C - STT-MRAM L2 Cache Analysis

Task 1: STT-MRAM vs. SRAM Baseline Comparison

The 2 MB STT-MRAM L2 cache was simulated in NVSim (45 nm, current-sensing read mode, 10 ns write pulse) and evaluated side-by-side against the updated Part B CACTI 6T SRAM baseline results.

Metric	6T SRAM Baseline (Part B)	STT-MRAM (Part C)	Shift / Comparison
Read Access Latency	2.902 ns	1.378 ns	−52.5% (2.1× faster)
Cycle Time	2.652 ns	–	–
Write Access Latency	2.902 ns	10.525 ns	+262.7% (3.63× slower)
Dynamic Read Energy	0.7929 nJ (792.89 pJ)	1.1180 nJ (1118 pJ)	+41.0%
Dynamic Write Energy	0.8512 nJ (851.22 pJ)	0.8830 nJ (883 pJ)	+3.7%
Total Leakage Power (per bank)	562.632 mW	699.920 mW	+24.4%
Total Cache Area	11.474 mm ² (2.213 × 5.184 mm)	4.574 mm ²	−60.1% (2.51× denser)

Table 6: Side-by-side performance comparison between 6T SRAM (CACTI) and STT-MRAM (NVSim) at 2 MB.

Key Physical & Circuit Level Insights:

- **Area Density:** STT-MRAM reduces the total cache silicon footprint from 11.474 mm² down to 4.574 mm² (60.1% space savings). Replacing 6-transistor SRAM cells with compact 1T1MTJ cells increases storage density dramatically.
- **Read Speedup via Interconnect Scaling:** Total data read access time drops from 2.902 ns down to 1.378 ns. In SRAM, H-tree input/output routing contributes 1.355 ns (0.850 ns in + 0.505 ns out) to total delay; STT-MRAM’s 2.5× area reduction shortens these global interconnect paths significantly.
- **Write Latency Trade-Off:** STT-MRAM write delay (10.525 ns) is 3.63× slower than SRAM (2.902 ns) due to the continuous 10 ns spin-transfer torque current pulse required to overcome MTJ thermal stability (E_b) and flip free-layer polarization.
- **Energy Profile:** Dynamic read energy is 41.0% higher in STT-MRAM (1.118 nJ vs. 0.793 nJ) due to active current-sensing through the resistive MTJ stack, whereas dynamic write energy remains comparable (0.883 nJ vs. 0.851 nJ).

Task 2

Comparing the STT-MRAM L2 cache to the 6T SRAM baseline shows key operational trade-offs:

- **Area Reduction (4.574 mm² vs 11.47 mm², 60.1% smaller):** STT-MRAM uses a 1T1MTJ cell ($\sim 54 F^2$) instead of a 6-transistor SRAM cell ($\sim 146 F^2$). Replacing 5 transistors with 1 MTJ saves significant silicon area.
- **Faster Read Access (1.378 ns vs 2.902 ns, 52.5% faster):** Because the memory array is smaller, wordlines, bitlines, and H-tree routing wires are shorter, reducing wire resistance and capacitance delays.
- **Slower Write Access (10.525 ns vs 2.902 ns, 3.63× slower):** Flipping the MTJ magnetic layer requires passing current for a full 10 ns write pulse. SRAM writes much faster through inverter feedback.
- **Higher Read Energy (1.118 nJ vs 0.793 nJ, 41.0% higher):** STT-MRAM reads require active current flowing through the resistive MTJ stack to detect states, which burns more power than SRAM’s voltage sensing.

Task 3

Increasing the TMR ratio to 200% ($R_{ON} = 4 \text{ k}\Omega$, $R_{OFF} = 12 \text{ k}\Omega$) yields the following array-level shifts:

Metric	Baseline TMR 100%	Higher TMR 200%	Shift
Cache Hit Latency	1.378 ns	1.383 ns	+0.005 ns (+0.36%)
Cache Write Latency	10.525 ns	10.574 ns	+0.049 ns (+0.47%)
Bitline RC Charge Delay	257.92 ps	307.94 ps	+50.02 ps (+19.39%)
Dynamic Read Energy	1.118 nJ	1.115 nJ	−0.003 nJ (−0.27%)
Total Cache Area	4.574 mm ²	4.559 mm ²	−0.015 mm ² (−0.33%)

Table 7: Impact of TMR enhancement (100% vs. 200%) in NVSim.

Metric That Moved Most: The **Bitline RC Charge/Discharge Delay** changed the most, increasing by 19.39% (257.92 ps $\rightarrow$ 307.94 ps). Higher resistance in the MTJ cell (4 k Ω vs 3 k Ω) increases the overall bitline RC delay.

What Device-Level TMR Buys at the Array Level:

- **Sensing Reliability, Not Read Speed:** Higher TMR gives a larger difference between high and low resistance states ($\Delta I = I_{\text{read}} \cdot \frac{\Delta R}{R}$). This improves the sense amplifier’s read margin against process variations and noise without needing larger sensing circuits.
- **Array Latency Limit:** High TMR alone doesn’t speed up cache read time because H-tree routing and sense-amp timing dominate total access latency.

Task 4

Halving I_{Reset} from 200 μA to 100 μA with a fixed 54 F^2 cell area gives:

Metric	Baseline (200 μA)	Low Reset (100 μA)	Shift
Total Cache Area	4.574 mm^2	4.574 mm^2	0.000 mm^2 (0.0%)
Bitline & Cell Write Energy	0.265 pJ	0.140 pJ	-0.125 pJ (-47.17%)
Cache Write Dynamic Energy	0.883 nJ	0.883 nJ	0.000 nJ (0.0%)
Cache Read Latency	1.378 ns	1.378 ns	0.000 ns (0.0%)

Table 8: Impact of halving reset current (200 $\mu\text{A} \rightarrow 100 \mu\text{A}$).

Area Trend & Access Transistor Coupling:

- **Why Fixed Area Occurs:** In NVSim, fixing the cell area (54 F^2) in configuration overrides physical transistor scaling, keeping array size constant.
- **Transistor Sizing Link:** The access transistor width (W) must supply the peak write current (I_{write}):

$$I_{\text{write}} \leq I_{\text{drive}} = \mu C_{\text{ox}} \left(\frac{W}{L} \right) (V_{DD} - V_{th} - V_{\text{MTJ}}) \quad (2)$$

Halving I_{Reset} cuts local write energy by 47.17%. In custom layout designs, this lower current allows using a narrower transistor (W), shrinking cell area and bitline capacitance.

Comparative Summary (SRAM vs. STT-MRAM Variants):

Architecture	Area (mm^2)	Read Delay (ns)	Write Delay (ns)	Read Energy (nJ)
6T SRAM (Baseline)	11.470	2.902	2.902	0.793
STT-MRAM (Baseline)	4.574	1.378	10.525	1.118
STT-MRAM (200% TMR)	4.559	1.383	10.574	1.115
STT-MRAM (100 μA I_{Reset})	4.574	1.378	10.525	1.118

Table 9: Summary comparison across SRAM and STT-MRAM configurations.

- **Density & Speed:** STT-MRAM achieves a 60.1% smaller footprint and $2.1\times$ faster read access due to shorter array interconnects.
- **Trade-offs:** STT-MRAM trades write latency (10.525 ns) and read energy (1.118 nJ). Lowering I_{Reset} reduces local switching energy by 47.17%, helping mitigate write energy costs.

Part D - Ramulator 2.0: L2 Miss Stream on a DDR4 Channel

Methodology. An L2 miss trace containing 2,242,074 requests (524,448 reads and 1,717,626 writes) was collected from gem5 using a `CommMonitor` attached to the L2 memory-side port during a 32 MiB array benchmark (`TimingSimpleCPU`, 32 KiB L1I/L1D, 256 KiB L2). The protobuf trace was converted to Ramulator 2.0’s `LoadStoreTrace` format. The baseline uses a single DDR4 channel with the `FRFCFS` scheduler and `RoBaRaCoCh` address mapping. Each subsequent task changes only the specified parameter.

Task 1: Baseline DDR4 Channel Performance

Metric	Simulated Value
DRAM Architecture / Timing	DDR4, DDR4.3200AA
Scheduler / Address Mapping	FRFCFS / RoBaRaCoCh
Total Memory Cycles	16,700,347
Total Read / Write Requests	524,448 / 1,717,626
Average Read Latency	811.36 cycles
Average Write Latency	N/A (not tracked by controller)
Row Hits / Misses / Conflicts	982,768 / 14,951 / 1,244,295
Overall Row-Buffer Hit Rate	43.83%
Read-only Row-Buffer Hit Rate	92.93%
Write-only Row-Buffer Hit Rate	28.84%

Table 10: Baseline DDR4 performance and row-buffer statistics.

Latency Note. The `GenericDRAMController` does not track write completion latency. In addition, the printed average read latency uses a denominator that includes unsuccessful read-send attempts. Therefore, the average read latency reported above is recomputed using the accumulated read-latency sum divided by the 524,448 accepted reads:

$$\frac{425,515,229}{524,448} = 811.36 \text{ cycles.}$$

The row-event counters account for 2,242,014 requests, leaving a small 60-request accounting difference relative to the 2,242,074-request trace.

Key Observation. The overall row-buffer hit rate of 43.83% is mainly caused by the write traffic. Reads have a very high row-hit rate of 92.93%, whereas writes have only a 28.84% hit rate and approximately 70.5% row conflicts. Since writes constitute 76.6% of the requests, their poor row locality dominates the overall statistic. The high read latency is also consistent with significant memory-system congestion: the average read-queue length is 33.8 requests and the average total queue length is 55.2 requests.

Task 2: FCFS vs. FRFCFS Scheduling

To evaluate the effect of scheduling policy, a strict FCFS scheduler was added while keeping the trace, DDR4 configuration, and address mapping unchanged.

Metric	FRFCFS	FCFS	Impact
Total Memory Cycles	16,700,347	41,272,735	2.47×
Average Read Latency	811.36 cycles	2346.29 cycles	2.89×
Overall Row-Buffer Hit Rate	43.83%	44.34%	+0.51 percentage points

Table 11: Performance comparison: FRFCFS vs. FCFS.

Interpretation. The row-buffer hit rate remains almost unchanged because the underlying addresses and their bank/row locality are unchanged. However, FCFS cannot bypass an older request that is temporarily blocked by DRAM timing constraints. FRFCFS can instead select a younger request that is ready, particularly a request targeting an already-open row or another available bank. This allows FRFCFS to exploit bank-level parallelism and overlap otherwise exposed DRAM timing delays.

The average read latency increases by

$$2346.29 - 811.36 = 1534.93 \text{ cycles,}$$

corresponding to a **2.89×** increase. With $t_{RCD} = 22$ cycles and $t_{RP} = 22$ cycles, DRAM timing constraints can temporarily block older requests; however, the much larger observed latency increase is primarily due to queueing and scheduling effects rather than $t_{RCD} + t_{RP} = 44$ cycles alone.

Thus, the main benefit of FRFCFS is not a significant increase in the number of row hits, but more effective scheduling of requests that are ready to execute. This reduces both total memory-system execution time and average read latency.

Task 3: Row Bits Placed Below Bank Bits

To study the effect of address mapping, a custom `BaRoRaCoCh` mapper was implemented by swapping only the Row and Bank positions relative to the baseline `RoBaRaCoCh` mapping.

Metric	RoBaRaCoCh	BaRoRaCoCh	Impact
Total Memory Cycles	16,700,347	25,331,345	1.52×
Average Read Latency	811.36 cycles	1214.32 cycles	1.50×
Overall Row-Buffer Hit Rate	43.83%	43.79%	-0.04 percentage points

Table 12: Impact of swapping Bank and Row address-mapping bits.

Interpretation. The striking result is that the row-buffer hit rate is almost unchanged (43.83% versus 43.79%), yet execution time increases by 52%. Therefore, row-buffer hit rate alone does not explain memory-system performance.

The address mapping also determines how consecutive addresses are distributed across DRAM banks. The baseline `RoBaRaCoCh` mapping spreads consecutive accesses across banks more effectively, providing greater bank-level parallelism. Swapping the Row and Bank positions changes this distribution and increases contention within individual banks. Consequently, FRFCFS has fewer opportunities to find ready requests in other banks while a bank is waiting on timing constraints, increasing both total execution time and read latency.

Task 4: Doubling Channel Count

The final experiment doubles the number of independent DDR4 channels while keeping the workload and scheduling policy unchanged.

Metric	1 Channel	2 Channels	Impact
Total Memory Cycles	16,700,347	8,641,291	$1.93\times$ speedup
Average Read Latency	811.36 cycles	780.01 cycles	3.9% reduction
Channel Load Balance	100%	32.2% / 67.8%	Uneven distribution

Table 13: Performance scaling with two independent DDR4 channels.

Interpretation. Adding a second channel provides additional memory bandwidth and allows requests to be processed in parallel. The total execution time therefore falls from 16.70 million to 8.64 million memory cycles, corresponding to a $1.93\times$ speedup, close to the ideal $2\times$ scaling.

In contrast, average read latency improves by only 3.9%. This shows the distinction between *throughput* and *per-request latency*: additional channels increase the amount of memory traffic that can be processed concurrently, but they do not reduce the fundamental timing of an individual DRAM access.

The channel utilization is also uneven (32.2% / 67.8%), indicating that the address mapping does not distribute the trace perfectly evenly across the two channels.

Timing Observation. For the DDR4-3200AA configuration, the listed timing parameters include $t_{RCD} = 22$ cycles, $t_{RP} = 22$ cycles, $t_{CCD_S/L} = 4/8$ cycles, and $t_{RAS} = 52$ cycles. Although t_{RAS} is the largest of these parameters, it should not be interpreted as a 52-cycle minimum read latency. Instead, it constrains how long an activated row must remain active before it can be precharged. Actual request latency depends on row-hit status, bank availability, queueing, and the complete set of DRAM timing constraints.

Part E - System-Level Evaluation in gem5

Task 1: SRAM vs. STT-MRAM L2 Cache Performance

Workload	L2 Cache Type	simSeconds (s)	IPC	L2 Miss Rate
bfs	SRAM (2 MB)	0.023094	0.132221	1.044443
bfs	MRAM (8 MB)	0.023741	0.132191	1.015975
sssp	SRAM (2 MB)	0.027144	0.132077	1.053786
sssp	MRAM (8 MB)	0.027979	0.127745	1.022341

Table 14: System performance comparison under Deriv03CPU.

Performance Trends:

- **Runtime & Throughput:** SRAM achieves lower execution time and higher IPC across both graph workloads. Switching to STT-MRAM increases runtime by 2.80% on **bfs** and 3.08% on **sssp**.
- **Miss Rate vs. Hit Latency:** The 8 MB STT-MRAM cache reduces L2 miss rates by approximately 2.7–3.0%. However, the irregular memory-access behavior of these graph workloads makes them sensitive to cache-hit latency. The $2\times$ increase in L2 hit latency ($7 \rightarrow 14$ cycles) for STT-MRAM therefore hurts performance more than the modest reduction in cache misses helps.

Task 2: Trade-Off Analysis

Trade-off Winner: For the tested workloads, the trade-off does not favor the 8 MB STT-MRAM configuration; SRAM wins on both **bfs** and **sssp** due to its faster 7-cycle hit latency.

Key Drivers of Performance:

- **Capacity Diminishing Returns:** The irregular access behavior of these graph workloads means that quadrupling L2 capacity (2 MB $\rightarrow$ 8 MB) provides only a modest $\sim 3\%$ reduction in L2 misses, making the additional capacity insufficient to offset the higher hit latency.
- **Serial Dependency Chains:** Pointer-chasing accesses (e.g., `node[edge[i]]`) create long data dependencies in the CPU pipeline. Doubling the L2 hit delay adds stalls to these critical paths, lowering overall IPC.

Task 3: In-Order Core Evaluation (TimingSimpleCPU)

Workload	L2 Cache Type	simSeconds (s)	IPC	L2 Miss Rate
bfs	SRAM (2 MB)	0.081250	0.296866	0.130581
bfs	MRAM (8 MB)	0.082028	0.294050	0.130520
sssp	SRAM (2 MB)	0.098240	0.291164	0.130788
sssp	MRAM (8 MB)	0.099315	0.288012	0.127265

Table 15: System performance comparison under in-order TimingSimpleCPU.

Core Model Comparison:

- **Performance Gap Narrows:** SRAM remains faster than STT-MRAM under TimingSimpleCPU, but the slowdown drops from $\sim 3\%$ in O3 to approximately 1.0% (0.96% on **bfs** and 1.09% on **sssp**).
- **Impact of Out-of-Order Execution:** The O3 core can overlap independent instructions and therefore partially hide the additional L2 hit latency. However, serial pointer dependencies limit this latency hiding. In contrast, the in-order core has less opportunity to overlap independent memory operations, so execution is more directly serialized by memory dependencies. As a result, the relative SRAM–STT-MRAM performance gap is smaller in the measured TimingSimpleCPU runs.

Task 4: Cross-Layer Evaluation & Critical Review

(i) Overall Verdict: Does the Real Program Run Faster?

No. Replacing the 2 MB SRAM L2 cache with an area-equivalent 8 MB STT-MRAM cache does not improve execution speed for either tested workload. Although STT-MRAM reduces cache area by 60.1% (4.574 mm² vs. 11.470 mm²) and provides faster array-level reads (1.378 ns vs. 2.902 ns), the gem5 model uses a higher total L2 hit latency (14 cycles vs. 7 cycles), resulting in a 2.80–3.08% runtime increase. For these irregular, pointer-heavy graph workloads, the measured benefit from additional cache capacity is not sufficient to offset the higher modeled hit latency.

(ii) Reviewer Attack on Simulation Assumptions:

1. **Symmetric Read/Write Hit Modeling in gem5:** gem5 models L2 hits at a uniform 14 cycles, hiding the severe $3.63\times$ write latency penalty (10.525 ns) of STT-MRAM measured in NVSim. Ignoring the asymmetric read/write behavior and possible write-queue contention may overestimate STT-MRAM performance.
2. **Idealized Array Scaling and Thermal Assumptions:** NVSim assumes fixed room-temperature (27°C) conditions and idealized peripheral/layout scaling. SPICE simulations from Part A show that an 85°C temperature rise reduces read sensing margin by 20.25%. A reviewer could therefore question whether the simulated area and timing remain realistic under thermal and circuit-level constraints.
3. **Workload-Size Generalization:** The evaluation uses a single relatively small graph configuration, so the observed result may not generalize to larger graphs whose working sets exceed the cache capacity by a greater margin. A larger workload sweep would determine whether the $4\times$ capacity advantage of STT-MRAM eventually provides enough miss reduction to offset its higher L2 hit latency.

Code Repository

The complete source code and implementation files for this assignment are available in the following GitHub repository:

<https://github.com/nik2206/Amcas-Assignment1>